CAHIER DES CHARGES

Requirements Specification

SecureWaste

Sensitive Data Exposure Prevention Platform

Item	Value
Document	Cahier des charges — version 0.1 (Milestone 0)
Date	29 October 2026
Project type	Team portfolio project (5 members)
Institution	[to be completed]
Supervisor	[to be completed]
Team members & roles	[to be completed]
Status	Draft for review

Table of contents

1. General presentation 3
1.1 Context 3
1.2 The project 3
1.3 Purpose of this document 3
1.4 Disclaimer 3
2. Problem statement 4
3. Study of existing solutions 5
3.1 Method 5
3.2 Tools studied 5
3.3 Comparison 5
3.4 Critique 6
3.5 Positioning 6
4. Proposed solution and objectives 7
4.1 Solution overview 7
4.2 Objectives 7
4.3 Scope (MVP) and non-goals 7
5. Actors and use cases 8
5.1 Actors 8
5.2 Use case diagram 9
6. Functional requirements 10
6.1 Requirements table 10
6.2 Textual use case descriptions 11
7. Non-functional requirements 13
8. Static design 14
8.1 Data model 14
8.2 State machines 15
8.3 Class diagram 16
9. Dynamic design 17
9.1 End-to-end activity 17
9.2 Upload and scan 18
9.3 Review a finding 19
9.4 Redact a PDF 20
10. Architecture 21
10.1 Components 21
10.2 Deployment 22
10.3 Technical choices 23
11. Security requirements and threat model 24
11.1 Assets 24
11.2 Trust boundaries 25
11.3 STRIDE analysis (summary) 25
11.4 Accepted residual risks 26
11.5 Security test cases 26
12. Planning 27
13. Deliverables, constraints and risks 28
13.1 Deliverables 28
13.2 Constraints 28
13.3 Future direction (not committed) 28
13.4 Project risks 28
14. Glossary 29
References 30

1. General presentation

1.1 Context

Organisations exchange documents every day: invoices, contracts, payslips, customer lists, internal reports. Many of them contain data that must not leave a controlled audience — payment-card numbers, bank account identifiers, national identity numbers, credentials such as API keys, or documents explicitly labelled CONFIDENTIAL. A single mistaken attachment can expose this data.

Data Loss Prevention (DLP) is the family of tools that detect such sensitive content and help prevent its unintended disclosure. Enterprise DLP suites exist, but they are tied to large cloud ecosystems and are rarely accessible to small organisations or transparent enough for teaching.

1.2 The project

SecureWaste is a local web application, built in Python with Flask, that analyses a PDF or TXT document **before** it is shared. It detects potentially sensitive information, validates candidates where possible, estimates the risk with an explainable score, lets a human reviewer confirm or dismiss each finding, and produces a genuinely redacted copy of PDFs. It is developed by a team of five cybersecurity students as a portfolio project.

1.3 Purpose of this document

This cahier des charges defines **what** SecureWaste must do and **under which constraints**, and it presents the design chosen to achieve it. It is the reference for planning, implementation, testing and evaluation. Every requirement carries an identifier (US-, FR-, NFR-, T-, ST-) that is reused in the repository documentation, GitHub issues and tests.

1.4 Disclaimer

> SecureWaste is an educational project. It produces **potential** findings that require human review, it will miss some sensitive data and flag some harmless data, and its risk score is an application-defined estimate. It does not certify compliance with GDPR, PCI DSS, Tunisia's Organic Law 2004-63 or any other regulation. Only synthetic, fictional data is used for development and demonstration.

2. Problem statement

Before sharing a document, an employee should know whether it contains sensitive data and what to do about it. Today, small organisations typically rely on manual reading, which is:

- **Slow** — a 30-page report cannot be reviewed line by line for every send.
- **Inconsistent** — two reviewers flag different things; nothing is recorded.
- **Unreliable for hidden data** — a card number in a table footer, an API key in an appendix or an ID number in metadata are easy to miss.
- **Unsafe at the remediation step** — drawing black boxes over text in a PDF leaves the text extractable; many real leaks happened this way.

Problem: How can a small team detect, explain and remove sensitive information from documents before sharing them, using a transparent tool that does not itself become a new place where that information leaks?

3. Study of existing solutions

3.1 Method

We selected five representative tools covering enterprise DLP suites, cloud detection services, an open-source PII framework and an open-source secret scanner. For each, we read the official documentation pages listed in the references and compared them against the needs of our target users (small organisations, local use, explainability, verified redaction).

3.2 Tools studied

Microsoft Purview Data Loss Prevention [1]

Purview DLP applies organisation-wide policies across Microsoft 365 services (Exchange, SharePoint, OneDrive, Teams), Office apps and managed Windows/macOS endpoints. Detection combines keywords and regular expressions with internal function validation, secondary matches near the primary match, and machine learning. Policy actions include warning users with policy tips, blocking sharing (with or without override) and quarantining items at rest; activity flows into the Microsoft 365 audit log and alert dashboards.

Google Cloud Sensitive Data Protection [2]

Google's service scans content through a cloud API for **infoTypes** (email addresses, card numbers, ID numbers, …). It offers built-in country-specific and global detectors, custom regex and dictionary detectors, checksum validation, hotword rules that raise or lower likelihood based on nearby context, and a likelihood level for every finding. The documentation itself notes that detectors are not perfectly accurate and cannot guarantee regulatory compliance.

Amazon Macie [3]

Macie discovers sensitive data in **Amazon S3** using managed data identifiers (financial data, credentials, passport numbers, …), custom regex identifiers with optional proximity keywords, and allow lists to suppress known values. Objects must be stored in supported S3 buckets and formats to be analysed; findings follow a standard schema for further processing.

Microsoft Presidio [4]

Presidio is an MIT-licensed open-source SDK for detecting and anonymising personal data in text and images. Its analysers combine patterns, checksums and context, and it uses trained machine-learning models; its documentation warns that there is no guarantee it will find all sensitive information. It is a library to integrate rather than an end-user application.

Gitleaks [5]

Gitleaks is an MIT-licensed open-source tool that detects hard-coded secrets (passwords, API keys, tokens) in Git repositories and files using configurable rules. It is focused on source code and is not designed for personal data in business documents.

3.3 Comparison

Criterion	Purview DLP	Google SDP	Amazon Macie	Presidio	Gitleaks	SecureWaste (target)
Type / licence	Commercial suite	Commercial cloud API	Commercial cloud service	Open source (MIT) SDK	Open source (MIT) CLI	Open source (AGPL-3.0 proposed) app

Criterion	Purview DLP	Google SDP	Amazon Macie	Presidio	Gitleaks	SecureWaste (target)
Where it runs	Microsoft 365 + endpoints	Google Cloud	AWS (S3)	Your code / container	Local or CI	Local machine
Data leaves the organisation?	Within M365 tenant	Sent to cloud API	Must be in S3	No (local)	No (local)	No (local)
PDF / TXT documents	Yes (M365 content)	Via API	Objects in S3	Text input (needs extraction)	Files / repos	Yes (text PDFs, TXT)
Detection methods	Regex, keywords, validation, proximity, ML	Built-in + custom, checksums, hotwords, likelihood	Managed + custom identifiers, allow lists	Patterns, checksums, context, ML	Configurable secret rules	Regex + Luhn/IBAN validators + context words
Explainable per-document risk	Policy matches & alerts	Per-finding likelihood	Findings + severity	Per-entity score	Per-finding rule	Weighted score with listed reasons
Human review workflow	Alerts / incidents	—	Findings management	—	—	Confirm / false positive / resolve
Verified PDF redaction	—	—	—	Anonymiser (text/images)	—	Yes, with re-extraction check
Suits a small team / teaching	Needs M365 licensing	Needs GCP account	Needs AWS account	Needs integration work	Code only	Yes — that is its purpose

"—" means the capability is not part of the documentation we reviewed; it does not mean the product could never support it.

3.4 Critique

- **Enterprise suites (Purview, SDP, Macie) are powerful but ecosystem-bound.** They assume the organisation already lives in one cloud, needs administrators to configure policies, and in the case of SDP and Macie requires the data to be sent to or stored in the provider's cloud. For a 20-person firm this is costly and opaque.
- **Open-source building blocks are not end-user products.** Presidio offers excellent detection primitives but no upload workflow, review state, audit trail or verified PDF redaction; Gitleaks covers secrets in code only.
- **Explainability is limited.** Most tools report per-finding confidence; few explain a single overall "is this document safe to share?" answer in plain language.
- **Redaction verification is rarely visible to the user.** None of the reviewed tools presents an explicit "we re-extracted the text and the value is gone" proof for a sanitized PDF.
- **Their accuracy far exceeds ours.** SecureWaste will detect fewer categories, with simpler rules, in fewer formats. Its contribution is transparency, a complete local workflow and educational value — not detection breadth.

3.5 Positioning

SecureWaste targets the gap between enterprise DLP and raw libraries: a **local, explainable, document-centric** workflow — upload, detect, assess, review, redact with verification, report — that a small team can run, read and audit end to end.

4. Proposed solution and objectives

4.1 Solution overview

A Flask web application with a modular processing pipeline: **upload validation → page-aware text extraction → rule-based detection with validators → masking → explainable risk scoring → persistence → review → verified redaction → HTML report**. Raw sensitive values never leave memory; the database stores masked values and keyed hashes only.

4.2 Objectives

ID	Objective	Success indicator
G-1	Detect the MVP categories in TXT and text-based PDF files	Detection tests on the synthetic corpus pass
G-2	Explain every finding and the overall risk	Each finding has explanation + recommendation; risk card lists weighted reasons
G-3	Never become a leak itself	No raw value in DB, logs, HTML or reports (automated test)
G-4	Keep a human in control	Every finding has a review status; no automatic deletion or sharing
G-5	Produce unrecoverable redactions	Re-extraction finds 0 selected values; original unchanged
G-6	Be reproducible and credible as a portfolio	README works on a clean machine; CI green; docs match code

4.3 Scope (MVP) and non-goals

In scope: PDF (text layer) and TXT; e-mail, phone, configurable keywords, payment cards (Luhn), IBAN (mod-97), secrets/tokens, confidentiality labels, Tunisian CIN (context-gated); risk score; dashboard; review; redaction; HTML report; audit log; single local user.

Not in scope: OCR, other formats, accounts and roles, AI classification, e-mail/endpoint/cloud integrations, automatic blocking, internet deployment. These appear in the *Future direction* (section 13.3).

5. Actors and use cases

5.1 Actors

Actor	Type	Description
Analyst	Primary, human	IT administrator or security analyst who uploads documents, reviews findings, redacts and deletes documents (personas Sana and Youssef).
Compliance officer	Secondary, human	Reads dashboards, findings and reports to verify that documents were reviewed (persona Leila). In the MVP she uses the same single local account.
File system / SQLite	System	Store originals, sanitized copies and masked results. Not actors in the UML sense but external resources.

5.2 Use case diagram

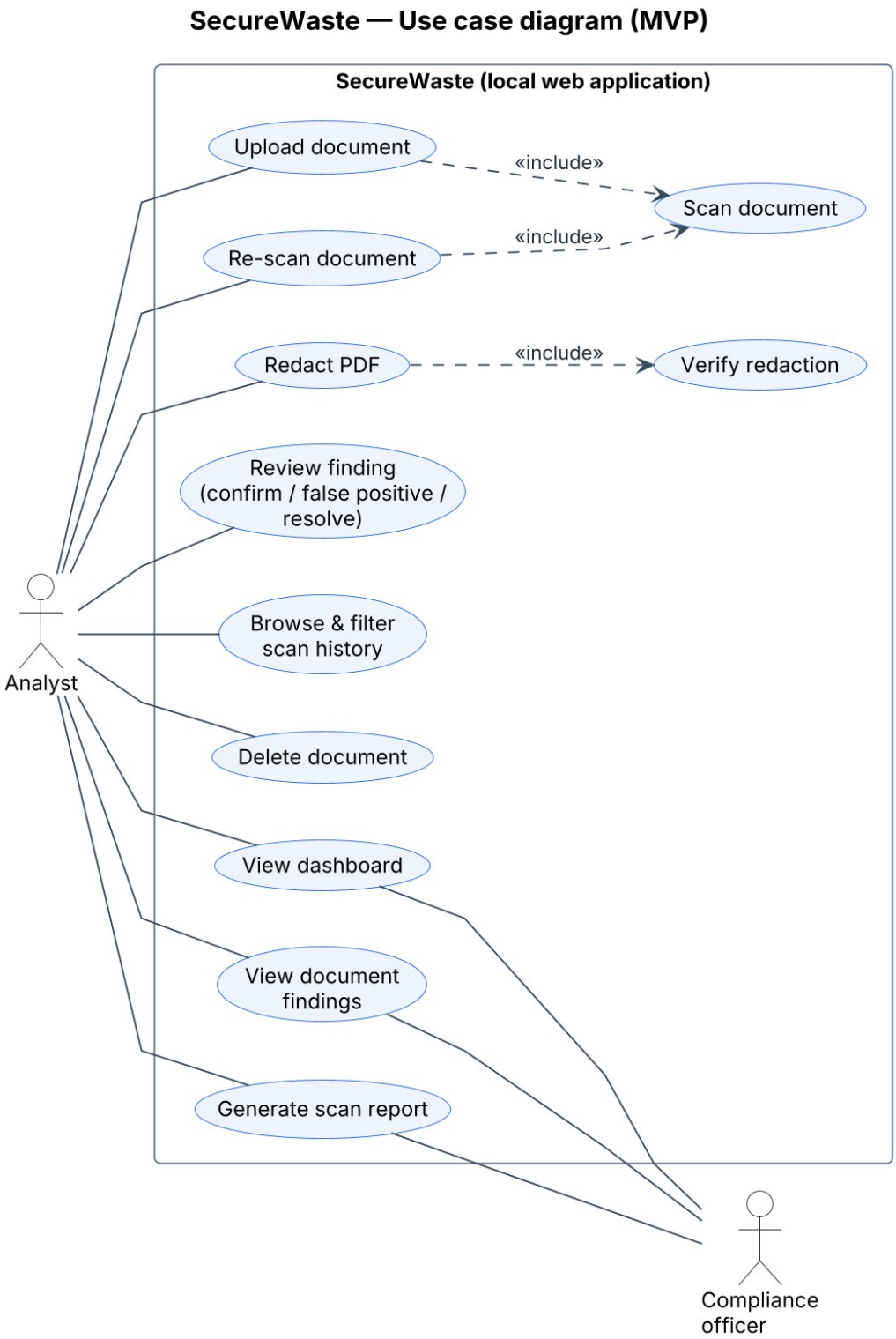


Figure 1 — Use case diagram (MVP)

How to read it. Stick figures are actors outside the system; ellipses inside the rectangle are goals they can achieve with SecureWaste. A plain line means the actor takes part in that use case. A dashed arrow labelled «include» means the first use case always performs the second one — uploading always triggers a scan, and redacting always triggers a verification.

6. Functional requirements

6.1 Requirements table

Priorities use **MoSCoW**: Must, Should, Could, Won't (this release).

ID	Requirement	Priority	Story
FR-01	Accept .pdf and .txt uploads and show accepted formats and size limit.	Must	US-01
FR-02	Reject files above MAX_UPLOAD_MB (default 10 MB) with HTTP 413.	Must	US-02
FR-03	Determine the real type from content (PDF magic bytes and successful open; TXT = UTF-8 without NUL bytes).	Must	US-02
FR-04	Reject encrypted, malformed, too long (>100 pages) or text-less PDFs with distinct error codes.	Must	US-02
FR-05	Store uploads under random server names outside the web root; keep the original name only as escaped display text.	Must	US-01
FR-06	Compute and store the SHA-256 of each upload.	Must	US-15
FR-07	Extract text page by page and keep page numbers; cap extracted characters.	Must	US-03
FR-08	Run every enabled detection rule; each returns category, page, offsets, confidence and rule ID.	Must	US-04
FR-09	MVP rules: e-mail, phone, keywords; payment card + Luhn; IBAN + mod-97; secrets; labels; TN CIN (context-gated).	Must / Should	US-04–07, 20
FR-10	Adjust confidence with validation and context; discard candidates below MIN_CONFIDENCE.	Must	US-05
FR-11	Merge duplicate matches into one finding with an occurrence count.	Must	US-08
FR-12	Attach severity, explanation and recommendation to every finding.	Must	US-08
FR-13	Mask values before they leave the detection engine; short masked context only.	Must	US-08
FR-14	Compute a 0–100 risk score from weights, confidence, repetition, labels and combinations.	Must	US-09
FR-15	Map the score to five levels with floors for validated secrets and cards.	Must	US-09
FR-16	Show the weighted reasons; label the score an application-defined estimate.	Must	US-09
FR-17	Exclude false positives and recompute the risk.	Must	US-13
FR-18	Persist documents, scans, findings, reviews, redactions and audit events in SQLite with foreign keys.	Must	US-10
FR-19	Dashboard: totals, risk distribution, findings by category, recent scans, unresolved findings.	Must	US-11
FR-20	Filter history by severity, category, review status and date.	Should	US-16
FR-21	Document page with metadata, scan status, risk card and grouped findings.	Must	US-12
FR-22	Change review status following the state machine; store each change.	Must	US-13
FR-23	Delete a document with its files, scans and findings; keep a value-free audit event.	Must	US-14
FR-24	Record audit events for upload, scan, review, redaction, report and deletion.	Must	US-15
FR-25	Apply genuine PDF redactions to selected findings, scrub metadata, save a separate __SANITIZED copy.	Should	US-17
FR-26	Verify redaction by re-extracting text and re-running the selected rules.	Should	US-17
FR-27	Report findings that cannot be located as unmappable; mark the redaction incomplete.	Should	US-18

ID	Requirement	Priority	Story
FR-28	Generate an HTML report with masked values only.	Should	US-19
FR-29	Re-scan a document as a new scan job.	Should	US-21

6.2 Textual use case descriptions

UC-01 — Upload and scan a document

Field	Description
Actor(s)	Analyst
Preconditions	The application is running on 127.0.0.1; the analyst has a PDF or TXT file.
Main success scenario	1. The analyst opens the upload page and selects a file. 2. The system checks the CSRF token and request size. 3. The system validates the real format and limits. 4. The system stores the file under a random name and creates a scan job (running). 5. The system extracts text per page, runs the detection rules, masks values and computes confidence. 6. The system computes the risk score and reasons, saves findings, marks the scan completed and records audit events. 7. The system shows the document details page.
Alternative flows	3a. Invalid file (too large, wrong type, encrypted, empty): the system shows an error code and message; nothing is stored. 5a. No text layer: the scan is marked failed with NO_TEXT_LAYER and the analyst is told OCR is not supported.
Postconditions	A document, a completed or failed scan, masked findings and audit events exist.

UC-02 — Review a finding

Field	Description
Actor(s)	Analyst
Preconditions	A completed scan with at least one finding.
Main success scenario	1. The analyst opens a finding and reads its masked value, explanation and recommendation. 2. The analyst selects confirmed, false positive or resolved and may add a note. 3. The system checks that the transition is allowed, stores the new status and a review action. 4. The system recomputes the document risk (excluding false positives) and records an audit event. 5. The page updates the finding badge and the risk card.
Alternative flows	3a. Transition not allowed (e.g. resolved → confirmed): the system answers 409 and nothing changes. 1a. Unknown finding ID: 404.
Postconditions	The finding has a new status, its history is recorded, and the risk reflects human judgement.

UC-03 — Redact a PDF

Field	Description
Actor(s)	Analyst
Preconditions	A completed scan of a PDF; the original file is still stored.

Field	Description
Main success scenario	1. The analyst opens the redaction page and selects findings. 2. The system re-scans the original in memory and matches candidates to the selected findings by page and keyed hash. 3. For each located value the system adds redaction areas and applies them, removing the underlying content. 4. The system removes metadata, annotations and embedded files and saves a new __SANITIZED.pdf. 5. The system re-extracts the text of the copy and re-runs the selected rules. 6. The system records the redaction (verified) and an audit event and offers the download.
Alternative flows	2a. A value cannot be located: it is listed as unmappable and the redaction is marked incomplete. 5a. A value is still present: the redaction is marked failed and the copy is not offered for download.
Postconditions	A separate sanitized copy and a redaction record exist; the original is unchanged.

UC-04 — Generate a scan report

Field	Description
Actor(s)	Analyst, Compliance officer
Preconditions	A completed scan.
Main success scenario	1. The user opens the report for a scan. 2. The system assembles metadata, counts by severity, risk score and reasons, recommendations, review statuses and redaction history — masked values only. 3. The user prints the page or saves it as PDF from the browser. 4. The system records an audit event.
Alternative flows	1a. Unknown scan ID: 404.
Postconditions	The user has a printable report.

UC-05 — Delete a document

Field	Description
Actor(s)	Analyst
Preconditions	The document exists.
Main success scenario	1. The analyst clicks Delete and confirms. 2. The system deletes the original and sanitized files, then the scans, findings, review actions and redactions. 3. The system records a document.deleted audit event without the filename. 4. The system returns to the history page.
Alternative flows	2a. A file is already missing: deletion continues and the event notes it.
Postconditions	No copy of the document or its findings remains in the application.

7. Non-functional requirements

ID	Category	Requirement
NFR-01	Security	No raw sensitive value is written to the database, logs, HTML, reports or audit events; only masked values and HMAC-SHA256 keyed hashes.
NFR-02	Security	CSRF tokens on all state-changing requests; HttpOnly and SameSite=Lax cookies.
NFR-03	Security	Strict Content-Security-Policy, nosniff, no-referrer, frame blocking; Jinja2 auto-escaping on.
NFR-04	Security	Binds to 127.0.0.1 by default; debug off; secrets from environment variables.
NFR-05	Security	Parameterised SQL only.
NFR-06	Performance	A 10-page text PDF is scanned in under 3 seconds on a typical student laptop.
NFR-07	Reliability	Malformed files never crash the app; temporary files are always removed.
NFR-08	Maintainability	Business logic in plain Python modules under core/, without Flask imports.
NFR-09	Testability	≥ 80 % line coverage on detection, risk and redaction modules; CI on every pull request.
NFR-10	Usability	Responsive from 390 px; WCAG 2.1 AA contrast; keyboard accessible; severity never shown by colour alone.
NFR-11	Portability	Windows, macOS, Linux with Python 3.12; no external services.
NFR-12	Privacy	Synthetic data only in the repository; secret scanning before release.
NFR-13	Transparency	Documented, versioned rule catalogue (RULESET_VERSION) stored with each scan.
NFR-14	Retention	Files kept until the user deletes them; no hidden copies; deletion is complete.

8. Static design

Static design describes what the system is made of — its stored data, the states its objects can be in and its classes — independently of time.

8.1 Data model

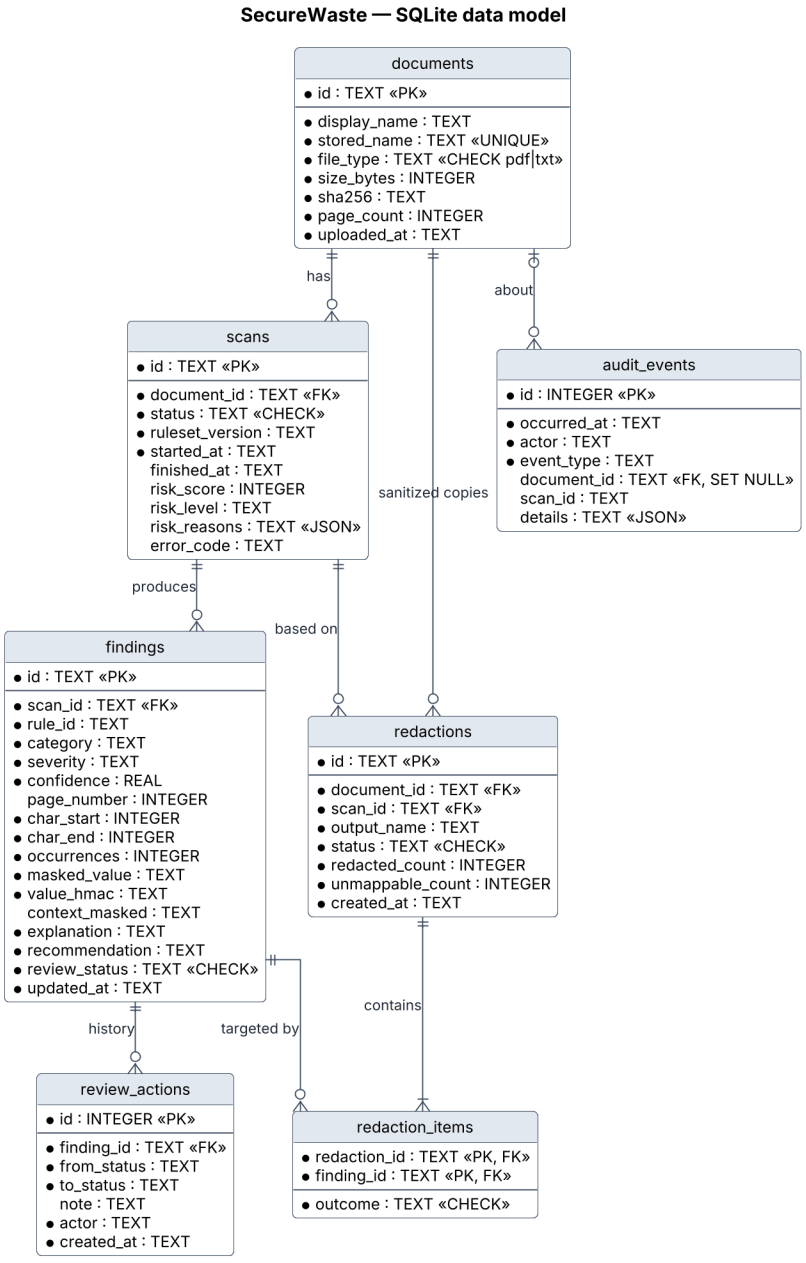


Figure 2 — SQLite data model

> **How to read it.** Each box is a database table with its columns; PK marks the primary key and FK a foreign key that points to another table. The symbols at the line ends give cardinality: two bars mean “exactly one”, a circle with a crow's foot means “zero or many”. For example, one document has zero or many scans, and each scan belongs to exactly one document.

8.2 State machines

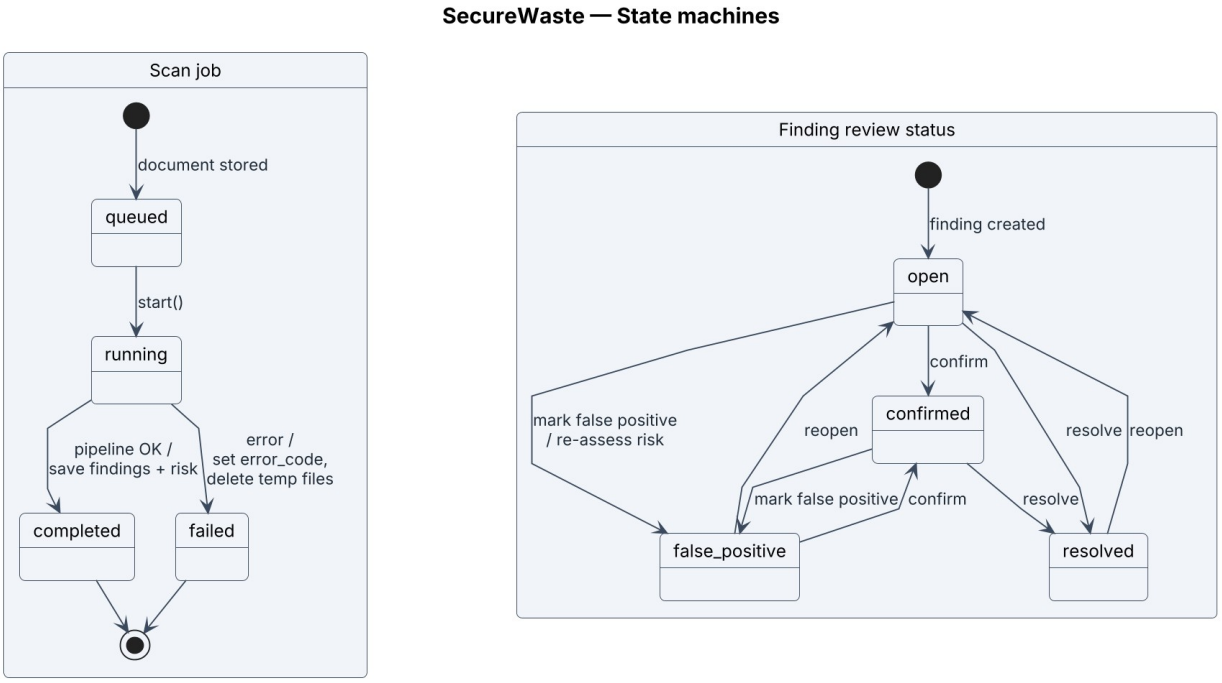


Figure 3 — Scan job and finding review state machines

How to read it. Rounded boxes are states and arrows are transitions labelled “event / action”. The black dot is where an object starts and the ringed dot where it ends. Read the right-hand machine as the rules of the review workflow: for instance a resolved finding can only be reopened, never directly marked as a false positive.

8.3 Class diagram

How to read it. Each box is a class: its name, then its attributes (data it holds), then its operations (what it can do). A filled diamond means composition — the whole owns the part and deleting a Document deletes its ScanJobs; a hollow triangle means inheritance (EmailRule is a RegexRule); a dashed arrow is a dependency such as “creates” or “uses”. Numbers like 1 and 0..* say how many objects can be linked.

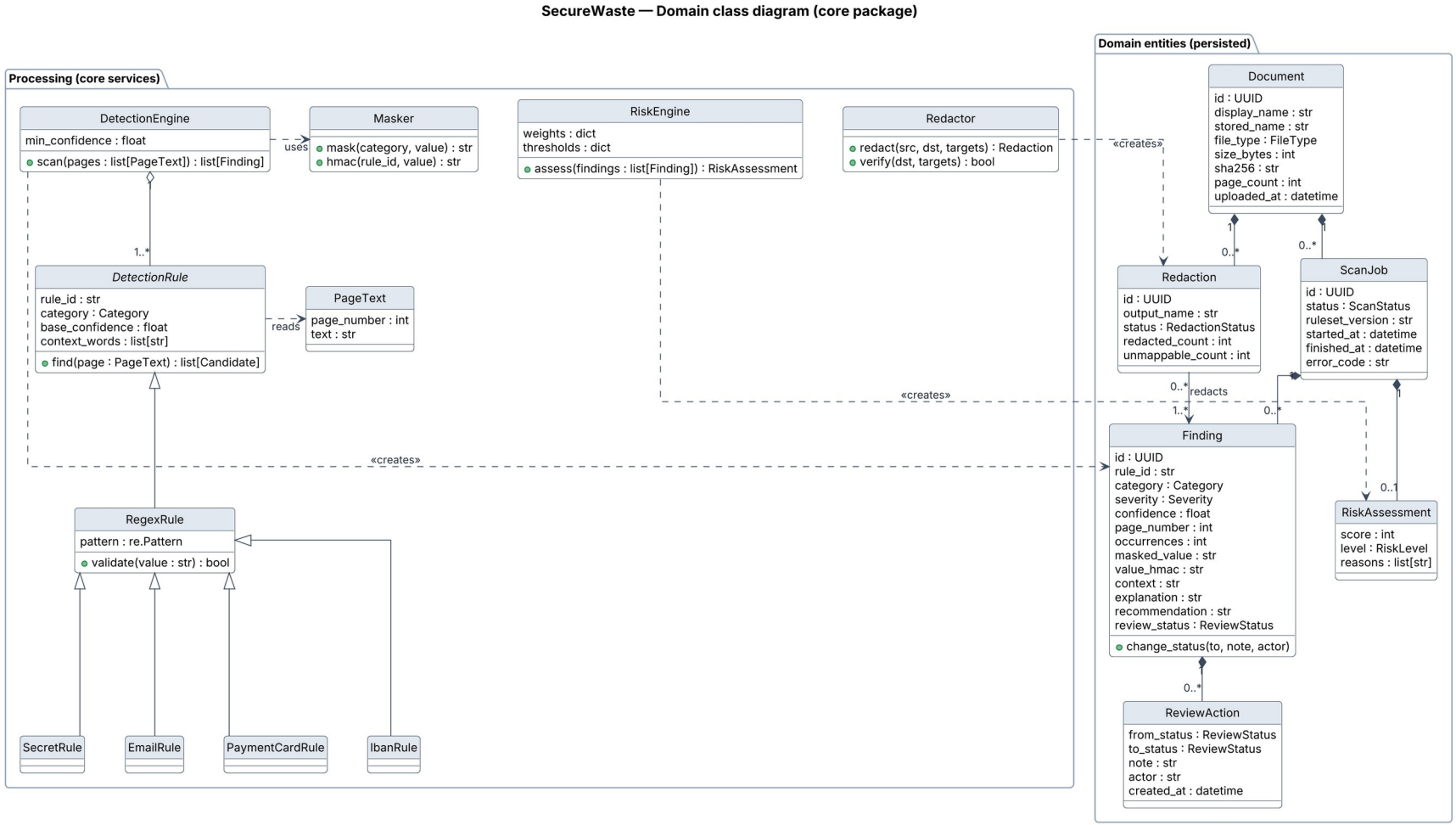


Figure 4 — Domain class diagram

9. Dynamic design

Dynamic design describes behaviour over time: first the whole workflow as an activity diagram, then the three main scenarios as sequence diagrams (each on its own landscape page so the messages stay legible).

9.1 End-to-end activity

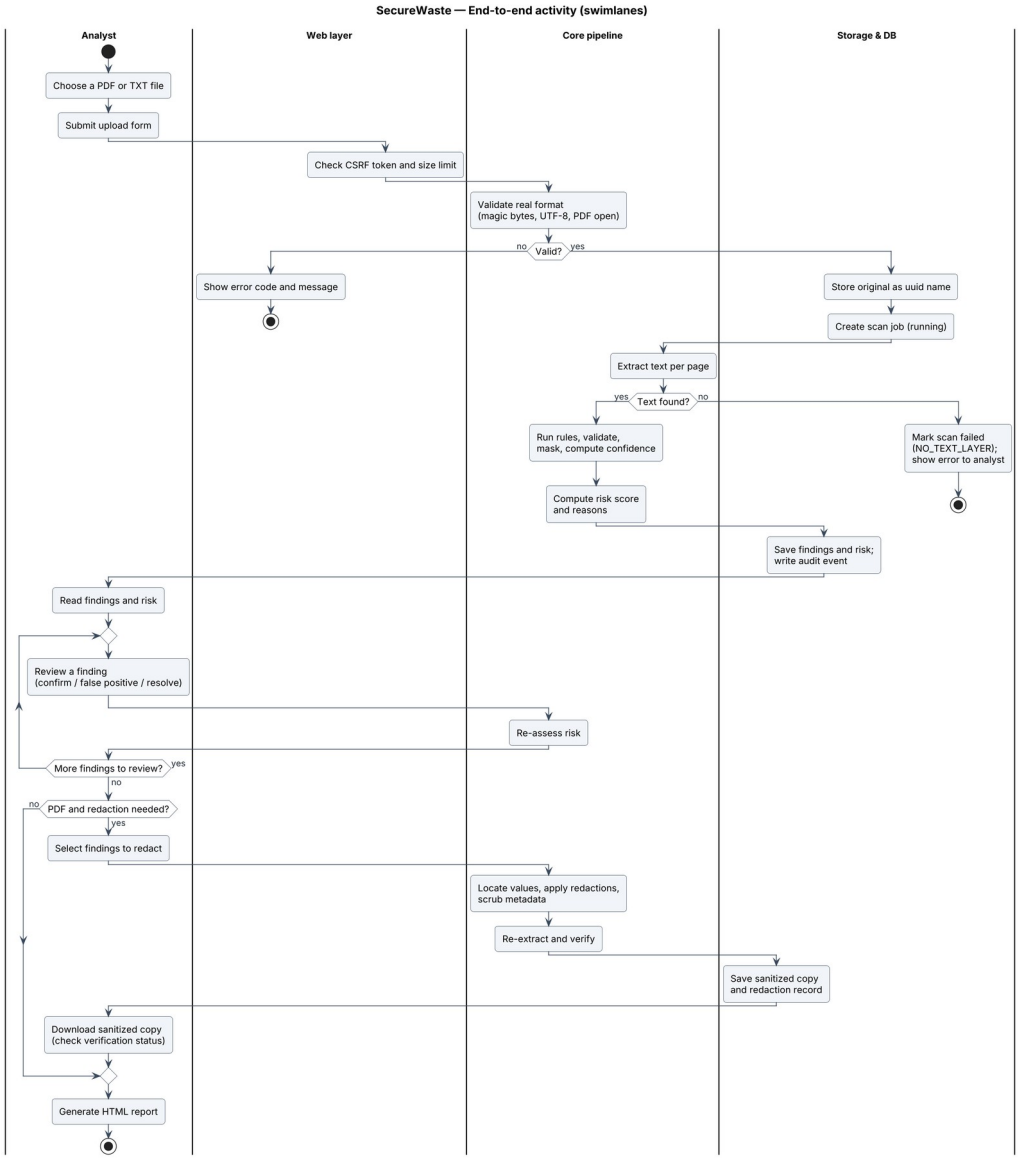


Figure 5 — Activity diagram with swimlanes

How to read it. Each vertical lane is a responsible party, so you can see who does what. Rounded boxes are actions, diamonds are decisions whose exits are labelled with a condition in square brackets, the black dot is the start and the ringed dots are possible ends. Follow the arrows from top to bottom to read the whole workflow.

9.2 Upload and scan

How to read it. Each column is a participant and time flows downwards. Solid arrows are calls, dashed arrows are returns, and the numbers give the order. The “alt” frame shows two alternatives (rejected or accepted file) separated by a dashed line; the “loop” frame repeats for every rule and page.

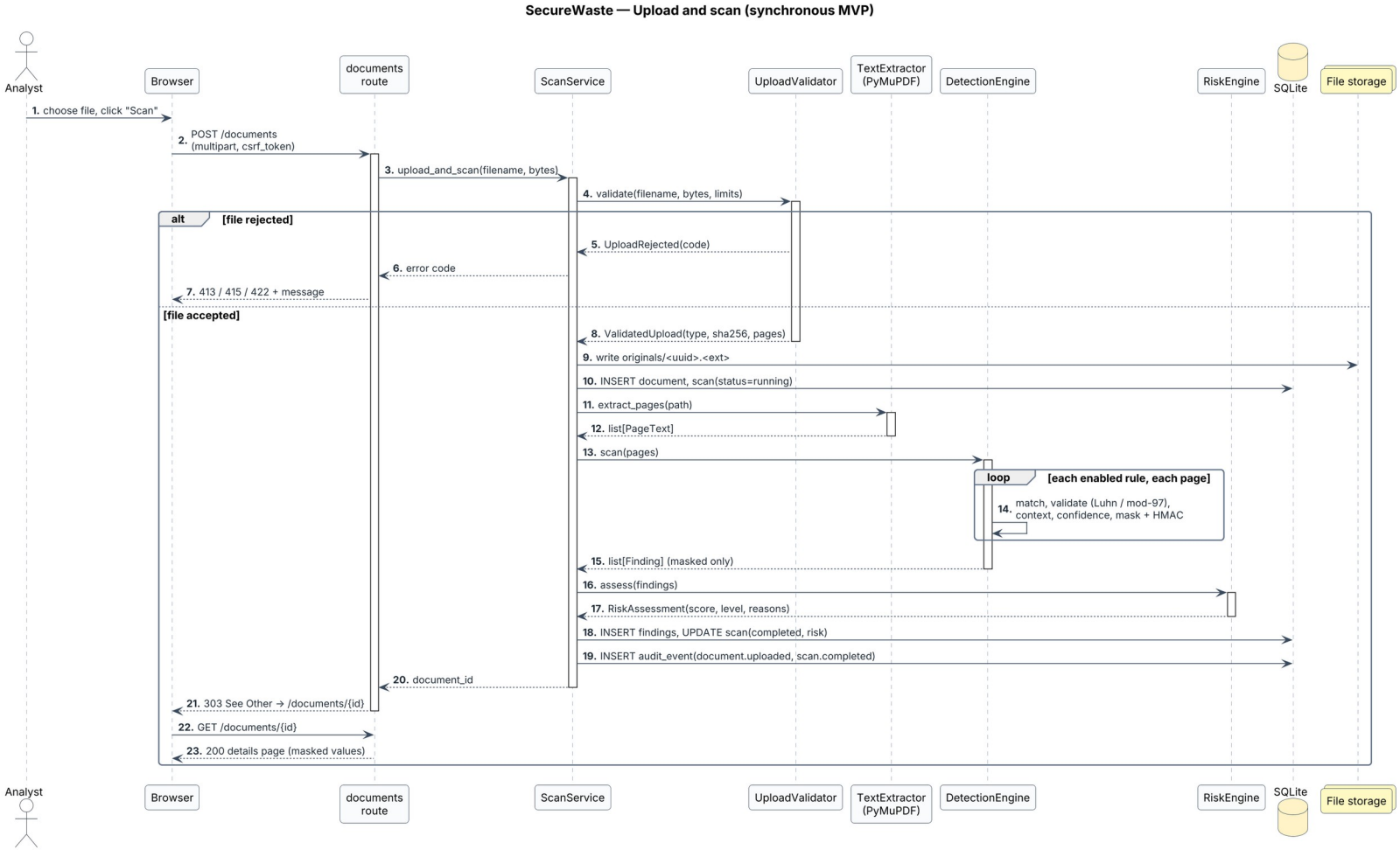


Figure 6 — Sequence diagram — upload and scan

9.3 Review a finding

How to read it. This diagram follows one JSON request from the browser. The alt frame lists the three possible outcomes — not found, transition not allowed, allowed — and only the last one changes data and recomputes the risk.

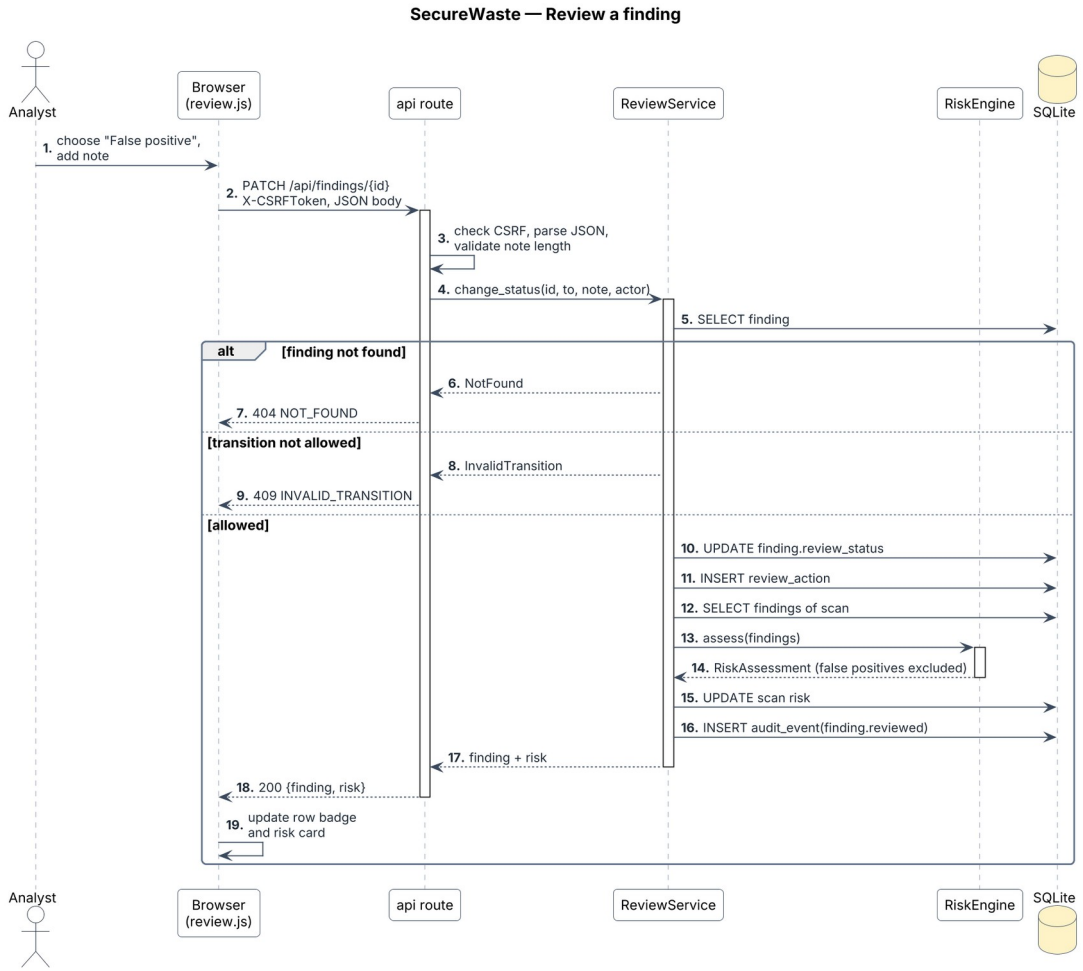


Figure 7 — Sequence diagram — review a finding

9.4 Redact a PDF

How to read it. Notice that the database is read for the selected findings but never for raw values: the service re-scans the original file in memory (steps 6–8) to recover them. Arrows that loop back to the same column are internal steps. The verification steps after the redaction are what allow the report to claim the copy is clean.

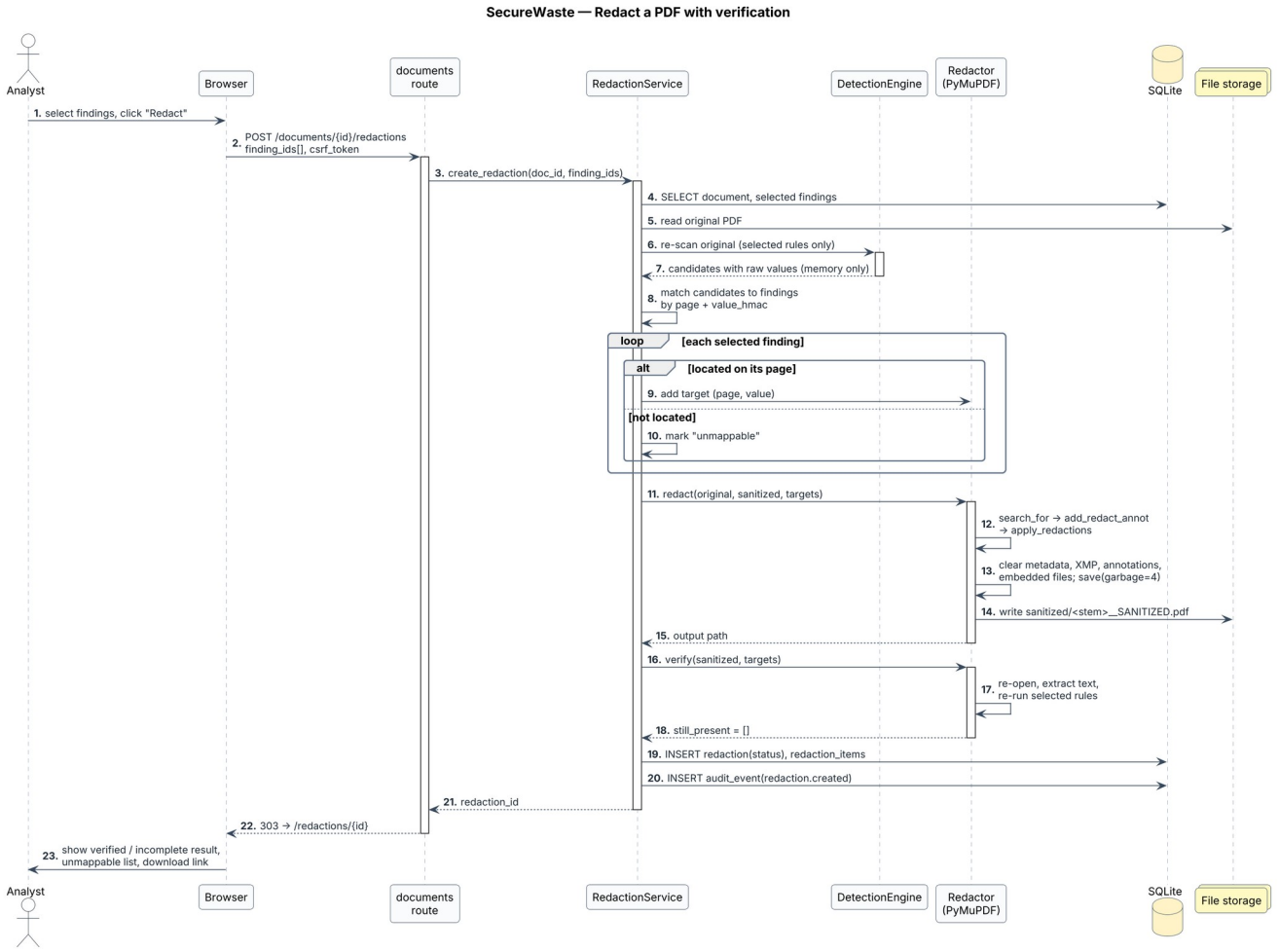


Figure 8 — Sequence diagram — redaction with verification

10. Architecture

The architecture is a layered monolith: one Flask process, one SQLite file, strict dependency direction web → services → core.

10.1 Components

How to read it. Boxes with the small tabs are components — replaceable parts with one responsibility — grouped in folders that are Python packages. An arrow from A to B means “A uses B”. The layers are strict: web calls services, services call core, and only services touch the database and file storage.

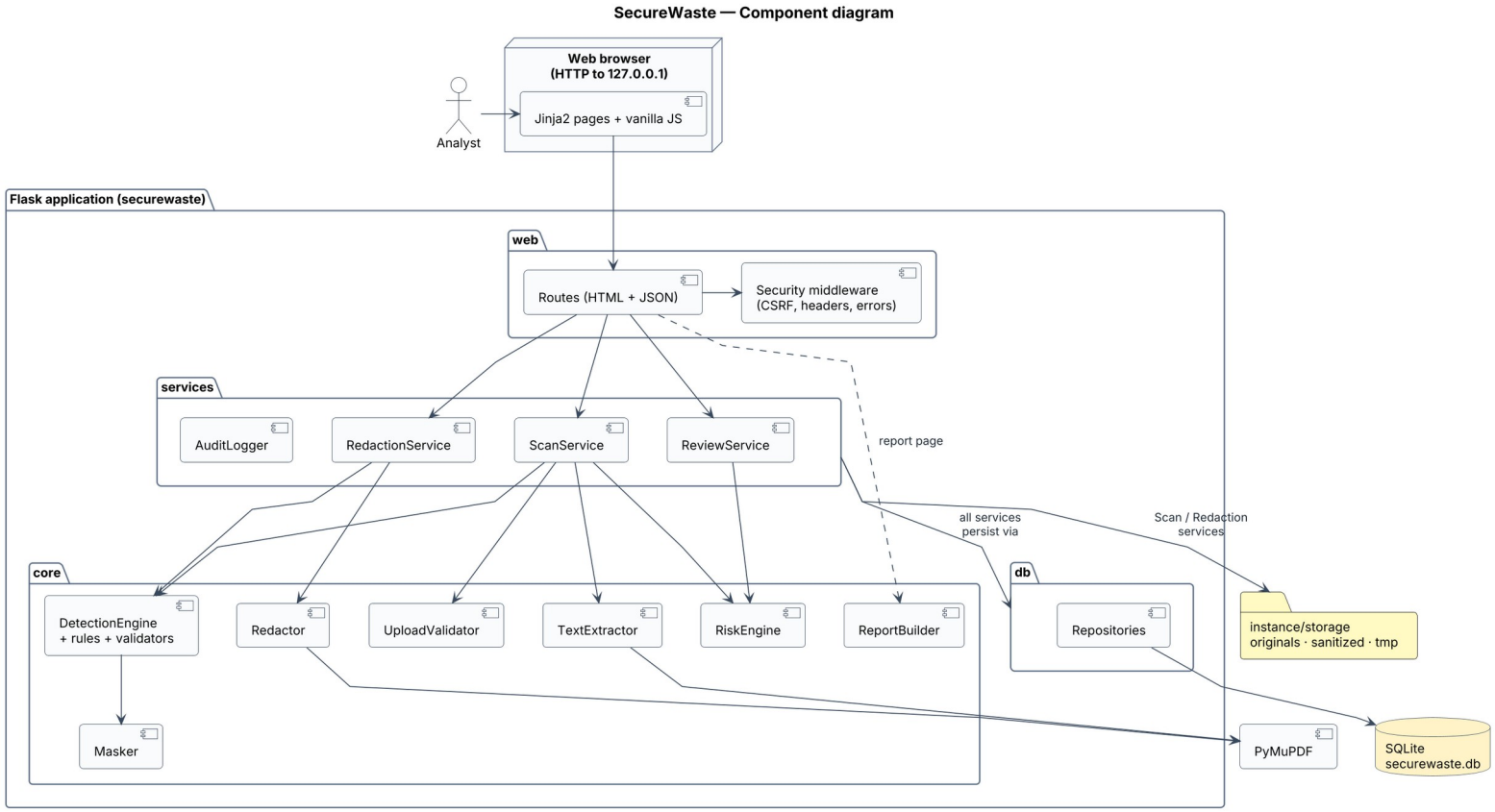


Figure 9 — Component diagram

10.2 Deployment

How to read it. 3-D boxes are nodes — machines or execution environments — and the folded-corner boxes are artifacts deployed on them. Lines are communication paths labelled with the protocol. Everything in the MVP runs on one laptop; GitHub only hosts code and runs tests on synthetic data.

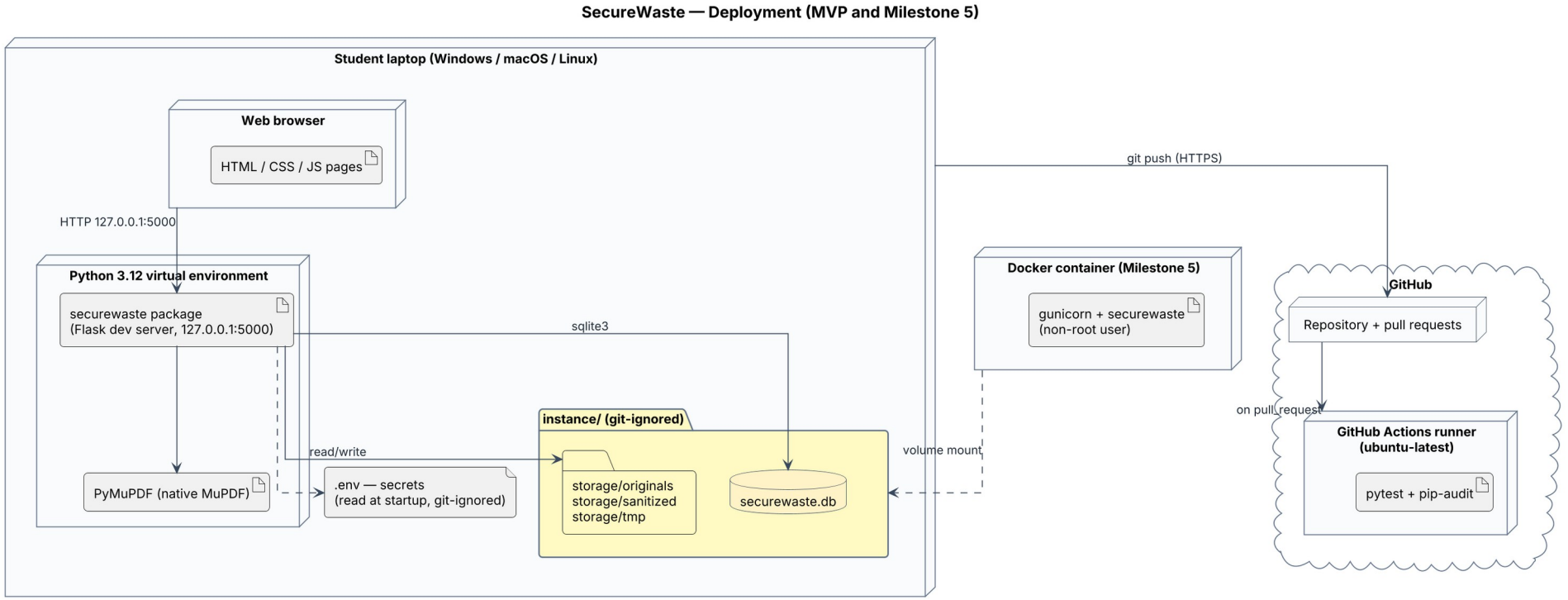


Figure 10 — Deployment diagram

10.3 Technical choices

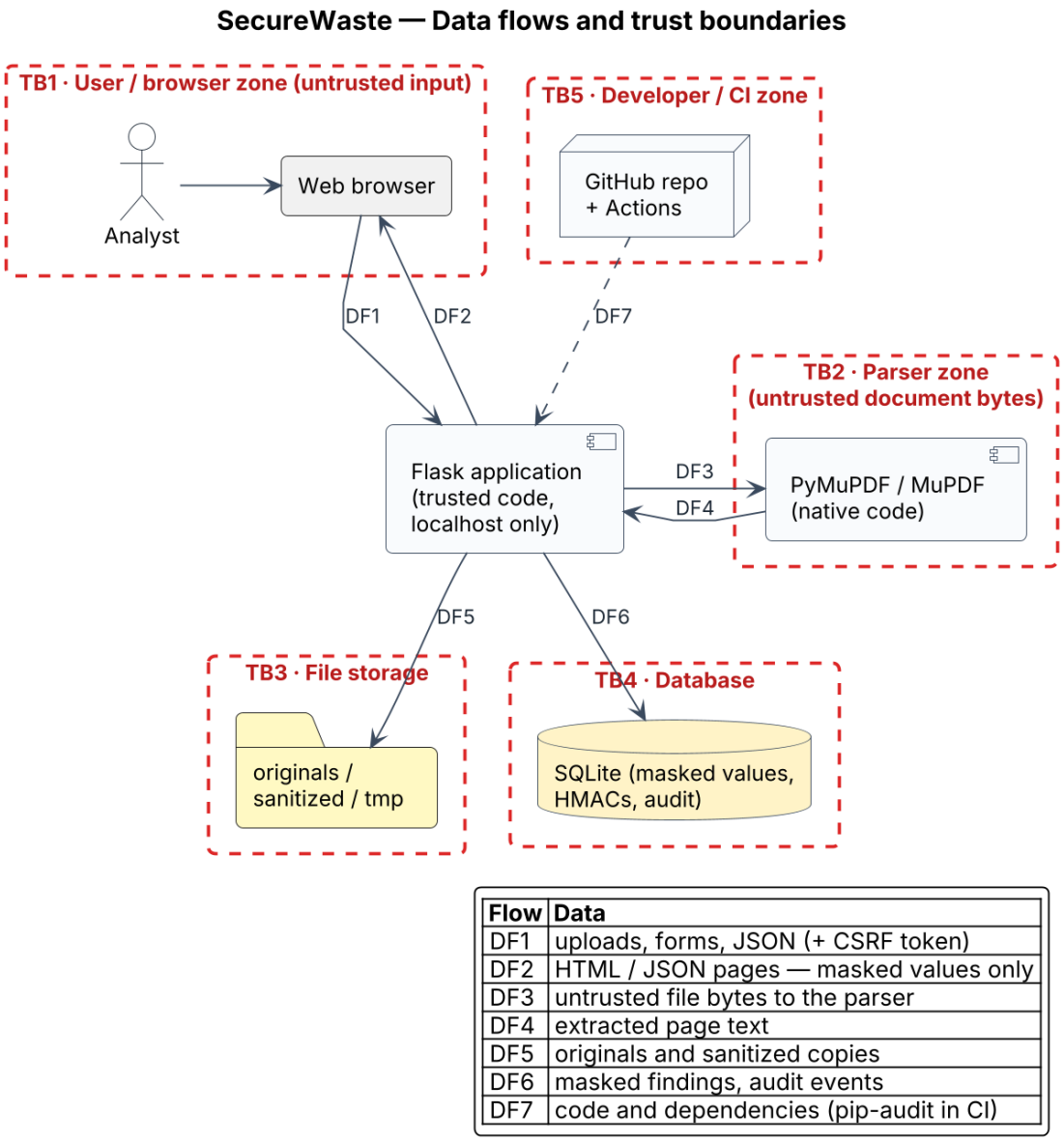
Need	Choice	Alternatives considered	Reason
Language	Python 3.12	Java, JavaScript/Node	Rich document and security libraries; readable for a learning team
Web framework	Flask 3 + Jinja2	Django, FastAPI + SPA, Streamlit	Small and explicit; strong XSS/CSRF defaults; no build step (ADR-0002)
PDF processing	PyMuPDF	pypdf, pdfplumber, pikepdf	Extraction with positions and genuine redaction in one library; AGPL licence (ADR-0004)
Detection	Regex + validators + context	Presidio, ML/LLM classifier	Explainable, testable baseline for later comparison (ADR-0005)
Database	SQLite via sqlite3	SQLAlchemy, PostgreSQL	Zero setup, real SQL, constraints (ADR-0003)
Sensitive storage	Masked values + HMAC	Raw values, encryption, plain hash	A stolen DB alone reveals nothing usable (ADR-0006)
Scan execution	Synchronous with job states	Celery/RQ queue, threads	Simple now, worker-ready later (ADR-0007)
Risk model	Weighted additive score	Count, max severity, ML, CVSS-like	Transparent and tunable (ADR-0008)
Front end	HTML, CSS, vanilla JS	React, Angular	No build tooling; design tokens from the Figma design system
CSRF / forms	Flask-WTF	Hand-rolled tokens	Proven implementation
Tests / CI	pytest + GitHub Actions + pip-audit	unittest, other CI	De-facto standards, free for public repos
Packaging	Docker (M5)	Plain venv only	Reproducible demos

11. Security requirements and threat model

11.1 Assets

ID	Asset	Why it matters
A1	Original uploaded documents	Contain the sensitive data
A2	Sanitized copies	Users will share them believing they are safe
A3	Findings (masked values, hashes, context)	Reveal where sensitive data is
A4	Audit trail	Accountability and evidence of review
A5	Secret keys (.env)	Session/CSRF integrity and hash resistance
A6	Availability	A result is needed before sharing
A7	Source repository and CI	Integrity of the code everyone runs

11.2 Trust boundaries



Flow	Data
DF1	uploads, forms, JSON (+ CSRF token)
DF2	HTML / JSON pages — masked values only
DF3	untrusted file bytes to the parser
DF4	extracted page text
DF5	originals and sanitized copies
DF6	masked findings, audit events
DF7	code and dependencies (pip-audit in CI)

Figure 11 — Data flows and trust boundaries

> **How to read it.** Dashed red rectangles are trust boundaries: data that crosses one must be checked, because the zone on the other side is less trusted. The labelled arrows DF1–DF7 are data flows, described in the legend. The most dangerous flow is DF3, where untrusted document bytes reach native parsing code.

11.3 STRIDE analysis (summary)

STRIDE classifies threats as Spoofing, Tampering, Repudiation, Information disclosure, Denial of service and Elevation of privilege. The full table (T-01 to T-21) is in docs/04-threat-model.md.

ID	STRIDE	Threat	Mitigation
T-01	S	Cross-site request forgery deletes documents or changes reviews	CSRF tokens, SameSite cookies
T-03	T	Path traversal through a crafted filename	UUID storage names; resolved path must stay in STORAGE_DIR
T-04	T	Fake redaction leaves text extractable	apply_redactions + re-extraction verification
T-05	T	SQL injection in filters	Parameterised queries, whitelisted values

ID	STRIDE	Threat	Mitigation
T-07	R	Reviewer denies a decision	Append-only review actions and audit events
T-08/09	I	Raw values in logs or DB	Masking + HMAC; no-leak tests
T-10	I	Stored XSS through filename or note	Autoescaping, CSP
T-11	I	Metadata survives redaction	Metadata, XMP, annotation and attachment removal
T-16/17	D	Huge upload or PDF bomb	Size, page and character limits
T-18	D	Regex catastrophic backtracking	Bounded patterns, adversarial tests
T-19	E	Memory-safety bug in the native parser	Updates, non-root container, future sandbox
T-20	E	Debugger enabled on a reachable host	Debug off by default; refuse debug on non-local host

11.4 Accepted residual risks

- No authentication in the MVP (single local user) — revisited at the accounts milestone.
- Short identifiers could be brute-forced from keyed hashes if both the database and the HMAC key are stolen.
- Context snippets may show undetected sensitive text next to a finding.
- Originals are stored unencrypted at rest; disk encryption is the user's responsibility.
- Pattern-based detection will miss some sensitive data (false negatives).

11.5 Security test cases

Fifteen security tests (ST-01 to ST-15) are defined in the threat model, for example: grep the database, logs and report for every raw fixture value (expect zero matches); upload ../../evil.pdf (expect a UUID name inside the storage folder); redact and re-extract with two different PDF tools (expect no selected value and empty metadata); post without a CSRF token (expect 400 and no change).

12. Planning



Figure 12 — Milestone roadmap (dates to be completed)

How to read it. Milestones are read from left to right; each card lists what must be demonstrably working at its end. Start and end dates are left blank on purpose so the team can set them with the supervisor. A milestone ends only when its acceptance criteria pass in CI.

Milestone	Content	Start	End	Exit criterion
M0	Requirements, architecture, threat model, ADRs, diagrams, repository workflow			Docs merged on main
M1	Flask skeleton, validated upload, extraction, first rules, CI			Upload → findings JSON in tests
M2	All MVP rules, validators, masking, confidence, risk score			Unit tests ≥ 80 % on detection/risk
M3	SQLite, history, dashboard, details, filters			Persisted scans visible in UI
M4	Review, recommendations, verified redaction, report, audit log			MVP acceptance criteria 1–8
M5	Security tests, retention, Docker, README, demo, presentation			Release v1.0.0 tagged

13. Deliverables, constraints and risks

13.1 Deliverables

- Source code in a public GitHub repository with CI.
- Repository documentation: README, requirements, architecture, interface contract, threat model, ADRs, diagrams (PlantUML and draw.io).
- This cahier des charges (Word and PDF).
- Automated test suite with synthetic fixtures and coverage report.
- Demonstration with fictional documents and a short recorded walkthrough.
- Portfolio presentation: problem, solution, architecture, security controls, limitations, future work.

13.2 Constraints

- **Technical:** Python, Flask, PyMuPDF, SQLite, vanilla front end; no microservices or heavy frameworks in the first version.
- **Legal / licensing:** PyMuPDF is AGPL-3.0; the project licence must be compatible (proposed AGPL-3.0).
- **Data:** only synthetic data; no real customer documents, identifiers or keys anywhere.
- **Team:** five students working part-time alongside courses; all work through issues and reviewed pull requests.
- **Scope honesty:** no claim of compliance or completeness.

13.3 Future direction (not committed)

- User accounts, roles and per-user document separation.
- Background job queue; parser isolation in a sandboxed subprocess.
- OCR for scanned PDFs; DOCX/XLSX/e-mail formats.
- Context-aware or AI-assisted classification, preferably local, measured against the rule-based baseline.
- Labelled benchmark corpus with precision, recall and false-positive rate.

13.4 Project risks

Risk	Likelihood	Impact	Mitigation
Scope creep (OCR, AI, accounts too early)	High	High	MoSCoW priorities; Future direction list; milestone reviews
Redaction cannot map some values	Medium	High	Report unmappable values; verification step; never claim success silently
Too many false positives annoy users	Medium	Medium	Validators, context gating, false-positive review, tuning per RULESET_VERSION
Uneven contributions in the team	Medium	Medium	Issues with owners, small PRs, weekly sync, review rotation
Real personal data committed by mistake	Low	High	.gitignore, fixture generator, secret scanning, PR checklist
Dependency vulnerability (PyMuPDF, Flask)	Medium	Medium	Pinned versions, pip-audit in CI, Dependabot
Learning curve delays M1	Medium	Medium	Learning guide, pair programming, smallest working slice first

14. Glossary

Term	Definition
ADR	Architecture Decision Record — a short document recording one technical decision and its reasons.
Audit event	A record that an important action happened (who, what, when), without sensitive values.
CSRF	Cross-Site Request Forgery — a malicious page making the user's browser send an unwanted request.
CSP	Content-Security-Policy — a response header telling the browser which scripts and resources it may load.
DLP	Data Loss Prevention — tools and practices that detect and prevent unintended disclosure of sensitive data.
False positive / negative	A harmless value wrongly flagged / a sensitive value missed.
HMAC	Hash-based Message Authentication Code — a hash computed with a secret key, so it cannot be recomputed without the key.
IBAN	International Bank Account Number; its last check is the mod-97 calculation.
Luhn checksum	A simple check-digit algorithm used by payment-card numbers to catch typing errors.
Masking	Replacing most characters of a value with symbols so it can be displayed safely.
MoSCoW	Prioritisation: Must, Should, Could, Won't (this time).
Redaction	Permanently removing content from a document, not just hiding it.
STRIDE	Threat categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.
Trust boundary	A line in the system where data passes between zones with different levels of trust.
UML	Unified Modeling Language — standard diagram notations for software design.

References

Pages consulted on 29 October 2026.

[1] Microsoft Learn — Describe data loss prevention in Microsoft Purview. https://learn.microsoft.com/nb-no/training/modules/describe-purview-data-solutions/4-describe-data-loss-prevention

[2] Google Cloud — InfoTypes and infoType detectors (Sensitive Data Protection). https://docs.cloud.google.com/sensitive-data-protection/docs/concepts-infotypes

[3] AWS — Discovering sensitive data with Amazon Macie. https://docs.aws.amazon.com/macie/latest/user/data-classification.html

[4] Microsoft Presidio — package documentation. https://pkg.go.dev/github.com/microsoft/presidio

[5] Help Net Security — Gitleaks: open-source solution for detecting secrets in your code (2024). https://www.helpnetsecurity.com/2024/06/27/gitleaks-open-source-solution-detecting-secrets-in-code/

[6] Arch Linux package page — gitleaks (licence MIT). https://www.archlinux.org/packages/extra/x86_64/gitleaks

[7] PyPI — PyMuPDF (dual AGPL / commercial licence). https://pypi.org/project/PyMuPDF/1.19.6/